水果分拣

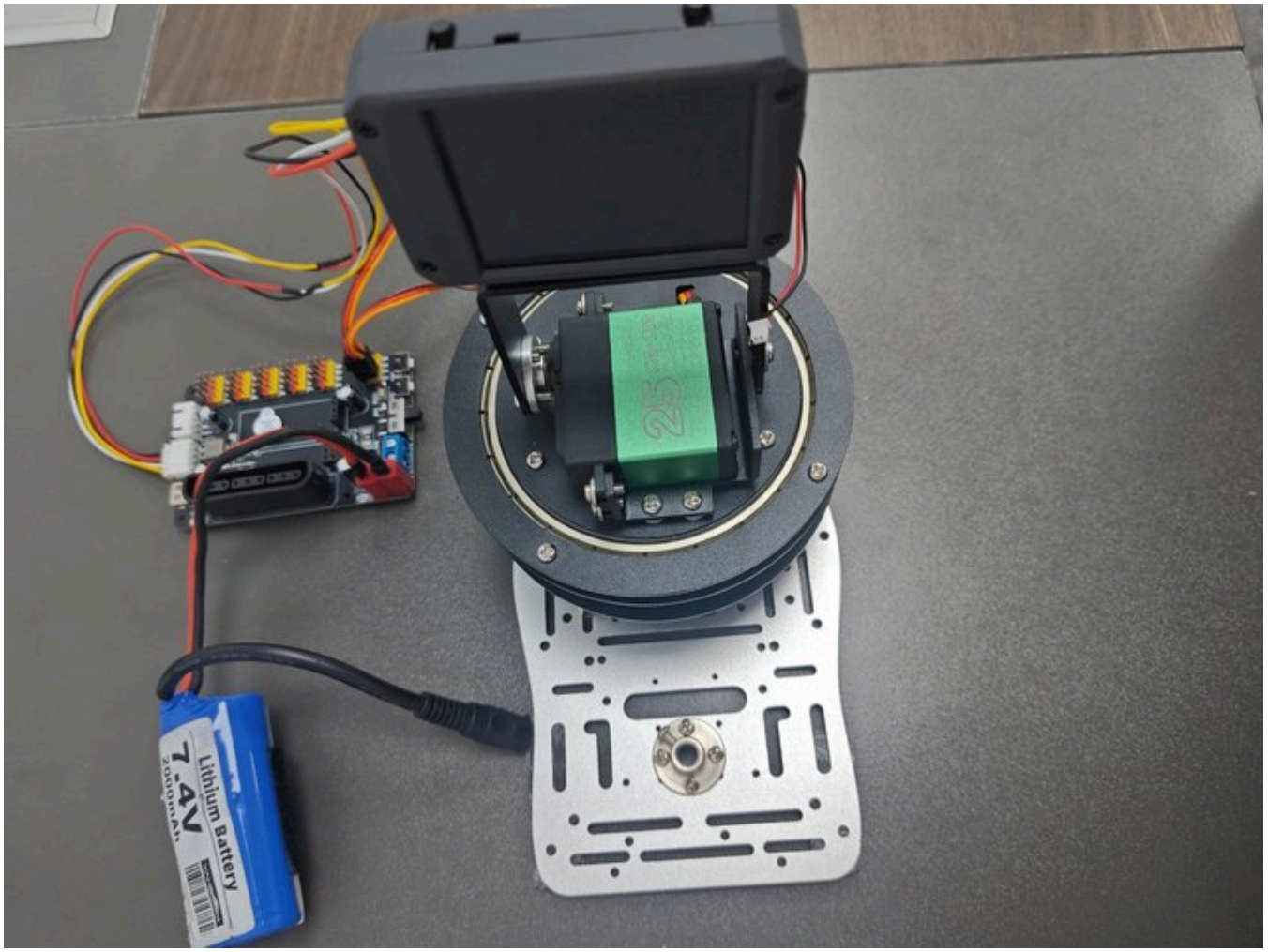
1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

2.器材清单

- k230 x1
- 24路舵机驱动板 x1
- 7.4v电池 x1 购买
- 铝合金板 x1 购买

3.摆放位置

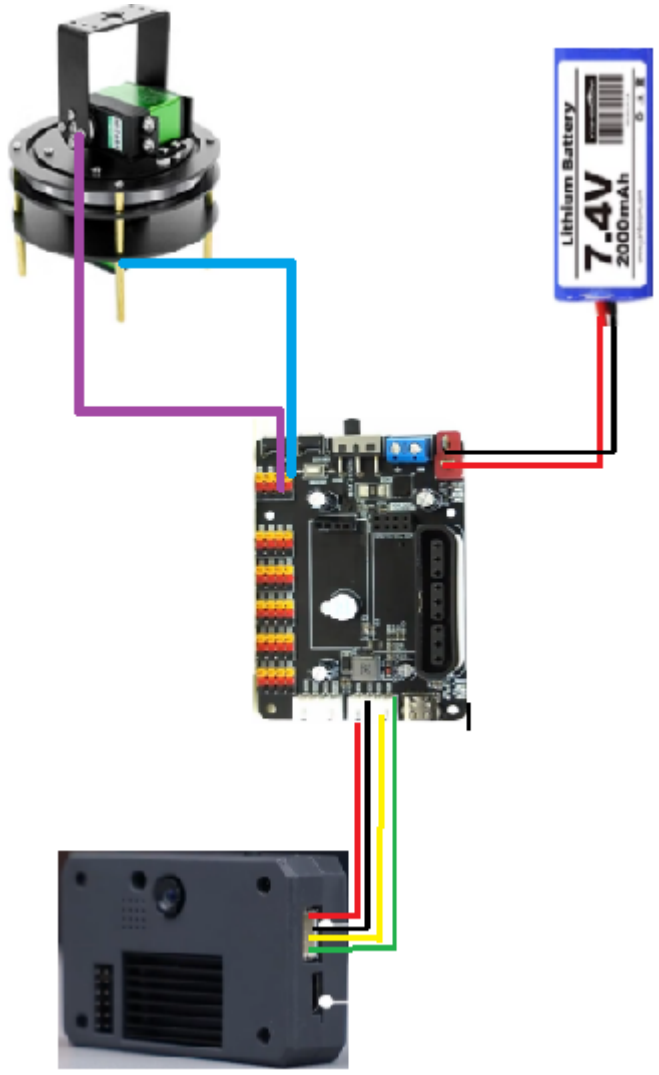


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行选择能固定的底板，底盘可以胶给粘住以免云台动的幅度大导致偏移，k230固定在二维云台上面,用电池的时候如果云台转动幅度过大可以适度减小pid参数。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

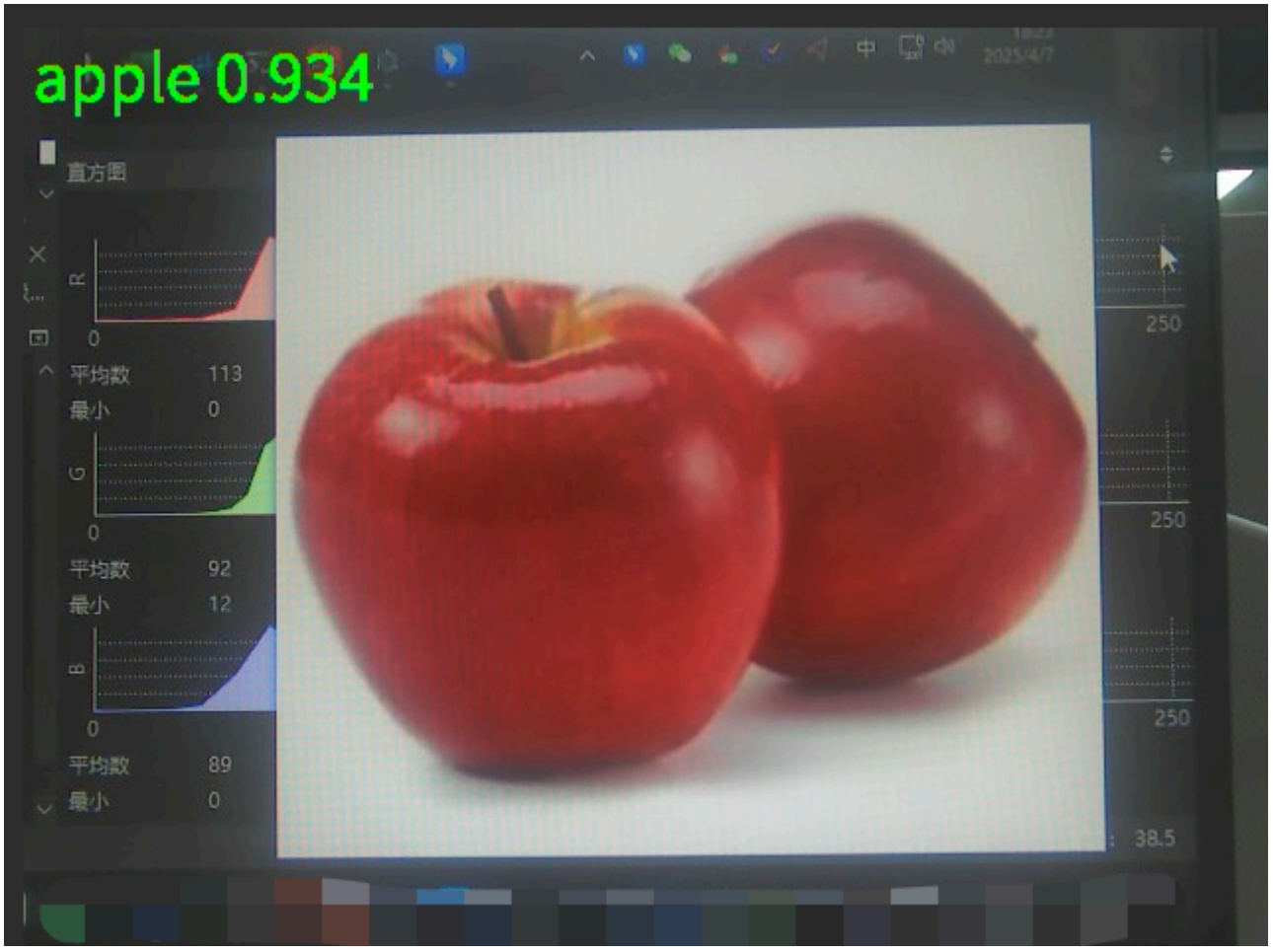
注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230关键代码解析

本节我们将使用基于yolov5训练的分类模型，来进行简单的水果分类

> 注：训练需要GPU环境，且具有一定的操作难度

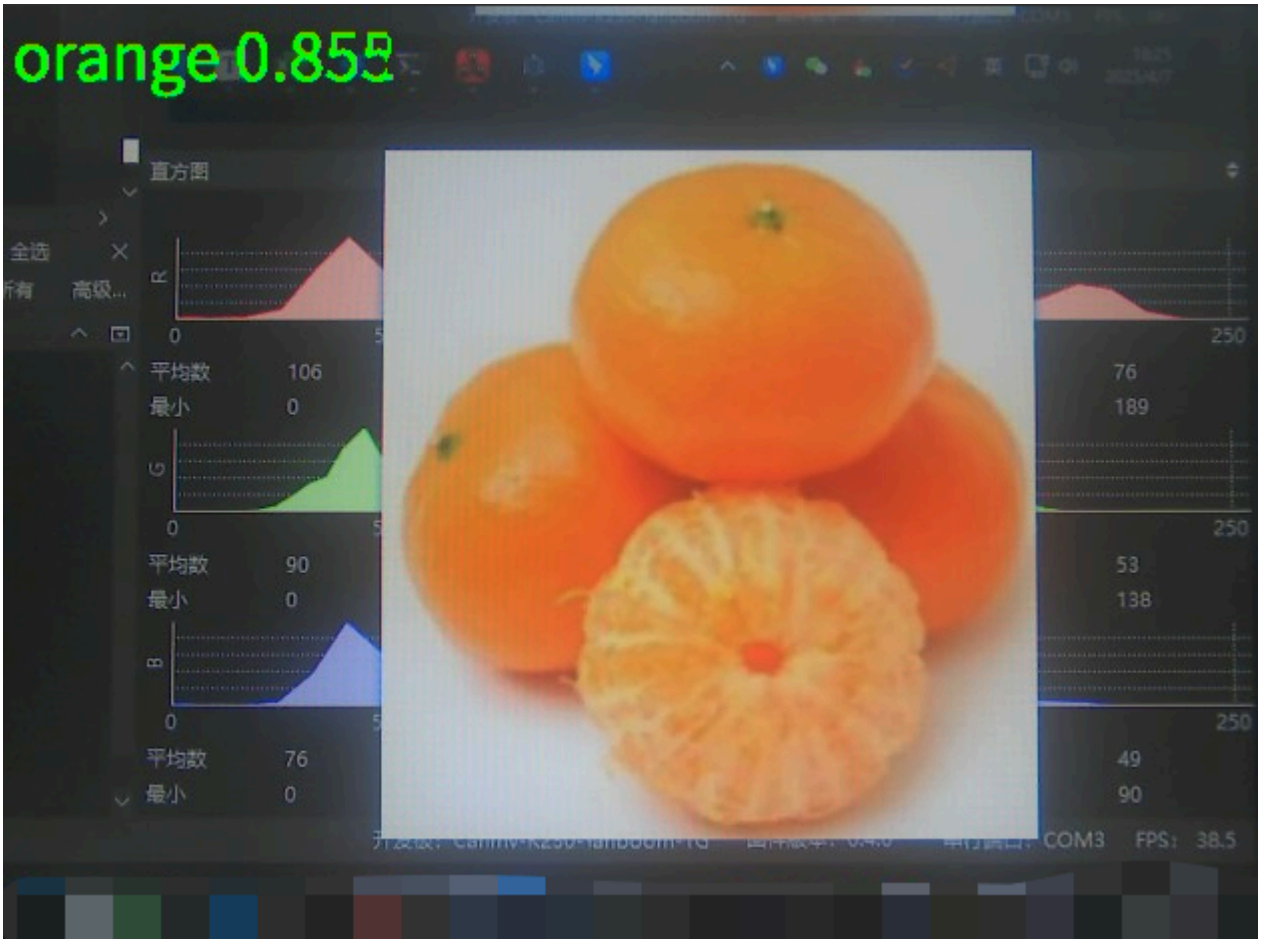
识别苹果



识别香蕉


banana 0.936

识别橘子


orange 0.85
直方图
全选
高级...
平均数 106
最小 0
平均数 90
最小 0
平均数 76
最小 0
76
189
53
138
49
90
COM3
FPS: 38.5

##

主要工作原理：

1. 从摄像头获取实时视频流
2. 对每一帧进行预处理，调整尺寸以适配模型输入要求
3. 将处理后的图像送入神经网络进行推理
4. 根据置信度阈值筛选分类结果
5. 在显示界面上绘制识别结果
6. 循环处理下一帧

导入依赖

```
from libs.PipeLine import PipeLine, ScopedTiming
from libs.YOLO import YOLOv5
import os,sys,gc
import ulab.numpy as np
import image
```

- `PipeLine`：处理图像流水线
- `ScopedTiming`：性能计时工具
- `YOLOv5`：封装了YOLO模型的推理接口
- `ulab.numpy`：MicroPython下的NumPy兼容库，针对嵌入式设备优化

配置初始化

```
rgb888p_size=[1280,720]  # 原始图像尺寸
display_size=[640,480]   # 显示尺寸
kmodel_path="/sdcard/kmodel/fruit_cls.kmodel"  # 模型路径
labels = ["apple","banana","orange"]  # 分类标签
confidence_threshold = 0.7  # 置信度阈值
model_input_size=[224,224]  # 模型输入尺寸
```

管道和模型实例化

```
pl=PipeLine(rgb888p_size=rgb888p_size,display_size=display_size,display_mode="lcd")
pl.create()
yolo=YOLOv5(task_type="classify",mode="video",kmodel_path=kmodel_path,labels=labels,
           rgb888p_size=rgb888p_size,model_input_size=model_input_size,
           display_size=display_size,conf_thresh=confidence_threshold,debug_mode=0)
yolo.config_preprocess()
```

主循环

```
try:
     while True:
            os.exitpoint()
            with ScopedTiming("total",1):
                # 逐帧推理
                img=pl.get_frame()
                res=yolo.run(img)
                flag=res[0]
                if flag == -1:
                    uart.write(bytearray([0x55,0xaa,0x00,0xfa]))
                if flag == 0:
                    uart.write(bytearray([0x55,0xaa,0xff,0xfa]))
                yolo.draw_result(res,pl.osd_img)
                pl.show_image()
                gc.collect()
```

5. 异常处理和资源释放

```
except Exception as e:
    sys.print_exception(e)
finally:
    yolo.deinit()
    pl.destroy()
```

6.stm32关键代码解析

- **control.c**

```
uint8_t get_active()
{
uint8_t lable ,ret=0;
    if(DataDecode2(&Uart3_RingBuff,&lable) == 0){
         if(lable == 0x00){
            rec_step_p[0][0] =1;
         }

         else if(lable ==0xff){
             rec_step_p[0][1] =1;

         }
    }

return ret;
}
```

此代码是将k230传过来识别水果的信息进行转换接收，然后赋值给标志位。

```
    {
    TIM_ITConfig(TIM1,TIM_IT_Update, ENABLE);
     TIM_Cmd(TIM4, ENABLE);
    USART_Cmd(USART3, ENABLE);
        get_active();

if(mode_flag==0&&rec_step_p[0][0]==1){


    USART_Cmd(USART3, DISABLE);
        go_to_point(0, 0, 400, 0);

        go_to_point(400, 0, 0, 0);
        rec_step_p[0][0]=0;

            mode_flag=1;

}

 if(mode_flag==0&&rec_step_p[0][1]==1){
        USART_Cmd(USART3, DISABLE);
        go_to(0, 0, 400, 0);
        go_to(400, 0, 0, 0);


            rec_step_p[0][1]=0;
                mode_flag=1;
}

}
```

此代码是在上面获得水果信息，然后控制舵机向一边移动然后返回。

```
void go_to_point(uint16_t now_x, uint16_t now_y, uint16_t targ_x, uint16_t targ_y)
{
    uint16_t i;
    float step_num = 200;/* 1000步*/
    float step_x, step_y; /* 步进值  */
    float going_x = now_x, going_y = now_y; /* 当前目标值*/

    step_x = (targ_x - now_x) / step_num;
    step_y = (targ_y - now_y) / step_num;

    for(i = 0; i < step_num; i++)
{

    kfp_x.source = step_x;
    kfp_y.source = step_y;
    kfp_x.out = KalmanFilter(&kfp_x, kfp_x.source);
    kfp_y.out = KalmanFilter(&kfp_y, kfp_y.source);
    going_x += kfp_x.out;
        going_y += kfp_y.out;
```

```
        set_target(going_x, going_y);
    }
}
```

此代码是将得到的两点坐标进行细分，从而是将矩形的2个点分成许多个点，然后许多点之间做pid运算，使得舵机运动更加精准。

```
void go_to(uint16_t now_x, uint16_t now_y, uint16_t targ_x, uint16_t targ_y)
{
    uint16_t i;
    float step_num = 100;/* 250²½ */
    float step_x, step_y; /* ²½½øÖµ  */
    float going_x = now_x, going_y = now_y; /* µ±Ç°Ä¿±êÖµ*/

    step_x = (targ_x - now_x) / step_num;
    step_y = (targ_y - now_y) / step_num;

    for(i = 0; i < step_num;i++){


            kfp_x.source = step_x;
            kfp_y.source = step_y;
            kfp_x.out = KalmanFilter(&kfp_x, kfp_x.source);
            kfp_y.out = KalmanFilter(&kfp_y, kfp_y.source);


            going_x += kfp_x.out;
      going_y += kfp_y.out;

      set_target(-going_x, -going_y);

    }
}
```

此代码是将得到的两点坐标进行细分，从而是将矩形的2个点分成许多个点，然后许多点之间做pid运算，使得舵机运动更加精准。

- ## bsp_servo.c

```
ServTypdef Xserv = {
    .pwm = 1600,
    .MAXPWM = 2400,
    .MINPWM = 500,
    .MIDPWM = 1400
};
```

```
ServTypdef Yserv = {
    .pwm = 1600,
    .MAXPWM = 2500,
    .MINPWM = 300,
    .MIDPWM = 1400
};
```

此代码是设置舵机启动的起始位置，所以需要修改这个来拟合云台运动的顶点，我将云台开机初始位置设置在中间点，通过这个改变舵机的起始位置。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }

    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                          //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- **pid.c**

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
             + PID->Kd * (PID->Err - PID->LastErr);                      /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出，如果想微调代码想达到更好的效果可以微调主函数的pid参数。

- **bsp_usart.c**

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {            //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){      //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){      //帧
尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
```

```
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

7.效果展示

通过yolov5训练模型识别水果，当k230识别到苹果时传输信息给二维云台向左运动然后返回，识别到橘子的时传输信息给二维云台向右运动然后返回，当你在云台上装个杆子的时候，左右运动可以达到分拣的效果。识别完一次之后要按下reset键，然后继续识别运动。